## Working on Bookworm Again

The Bookworm source contains very few unit tests, so it's up to you and Bill to write tests as you refactor. Sometimes this proves to be difficult, as is the case with this class:

class_definitions/bookworm_classvars.rb

```
class Loan
  def initialize(book)
    @book = book
    @time = Time.now
  end

  def to_s
    "#{@book.upcase} loaned on #{@time}"
  end
end
```

Loan stores the title of a book and the time when it was loaned—that is, the time when the object was created. You'd like to write a unit test for the to_s method, but to write that test, you'd have to know the exact time when the object was created. This is a common problem with code that relies on Time or Date: such code returns a different result every time it runs, so you don't know what result to test for.

"I think I have a solution to this problem," Bill announces. "It's a bit involved, so it will require some attention on your part. Here it is."

```
class Loan
  def initialize(book)
    @book = book
➤   @time = Loan.time_class.now
  end

➤ def self.time_class
➤   @time_class || Time
➤ end

  def to_s
    # ...
```

Loan.time_class returns a class, and Loan#initialize uses that class to get the current time. The class is stored in a *Class Instance Variable (109)* named @time_class. If @time_class is nil, the *Nil Guard (219)* in time_class returns the Time class as a default.

In production, Loan always uses the Time class, because @time_class is always nil. By contrast, the unit tests can rely on a fake time class that always returns